

Module 11: Intrusion Detection with Snort

Kayla Teekaram

Overview

This lab explored Snort, an open-source network intrusion detection system (IDS), by writing custom detection rules and then generating the exact traffic those rules were designed to catch. The goal was to go beyond the theory of signature-based detection and actually watch Snort fire alerts in real time against live attacker-style activity: reconnaissance (ICMP pings), credential leakage over FTP, unauthorized file access over HTTP, and a Netcat backdoor connection.

Environment

- IDS sensor: Ubuntu VM running Snort 2.9.x in console/IDS mode, monitoring interface ens33.
- Target host: 192.168.129.133, running FTP and HTTP services, with a Netcat listener configured as a simulated backdoor on port 4444.
- Attacker host: Windows machine used to generate traffic (ping, FTP client, nc).

Custom Rule Set

A local rules file (/etc/snort/rules/local.rules) was written with the following signatures, each targeting a distinct stage of attacker behavior:

```
alert icmp any any -> any any (msg:"ICMP PING Detected"; sid:100001; rev:1;)

alert tcp any any -> $HOME_NET 21 (msg:"Top secret questions leaked."; content:"RETR";
sid:100002; rev:1;)

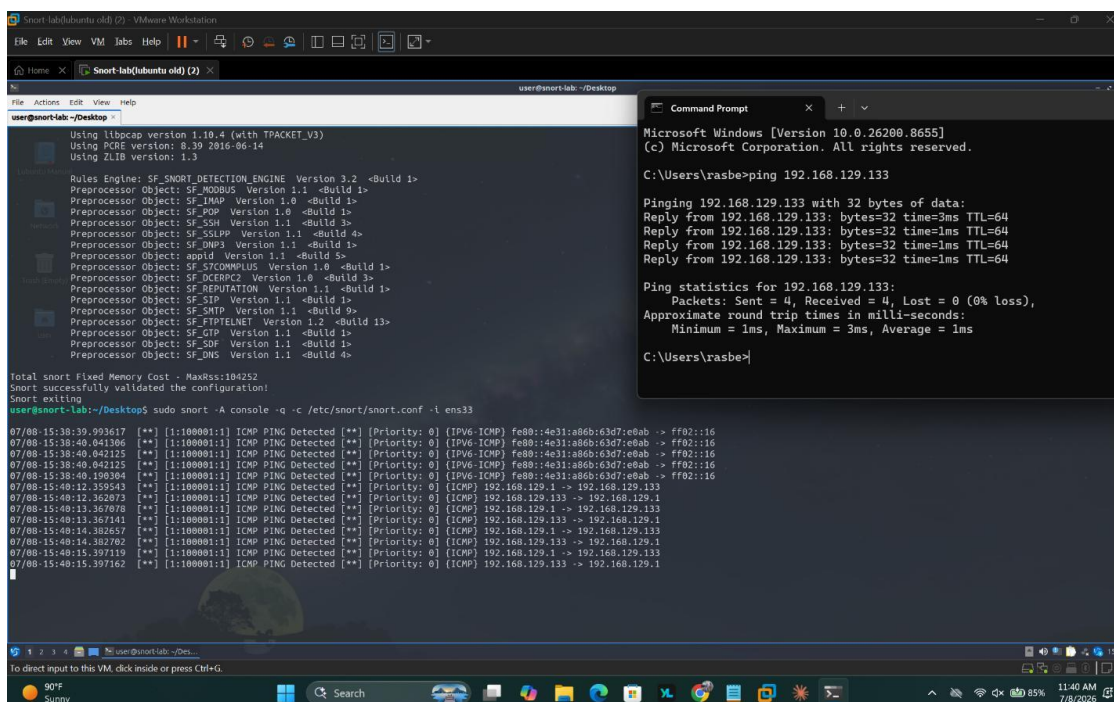
alert tcp any any -> $HOME_NET 21 (msg:"Unauthorized FTP server login attempt."; content:"USER
kayla"; nocase; sid:100003; rev:1;)

alert tcp any any -> $HOME_NET 80 (msg:"Secret-Plans.txt accessed."; content:"/Secret-
Plans.txt"; sid:100004; rev:1;)

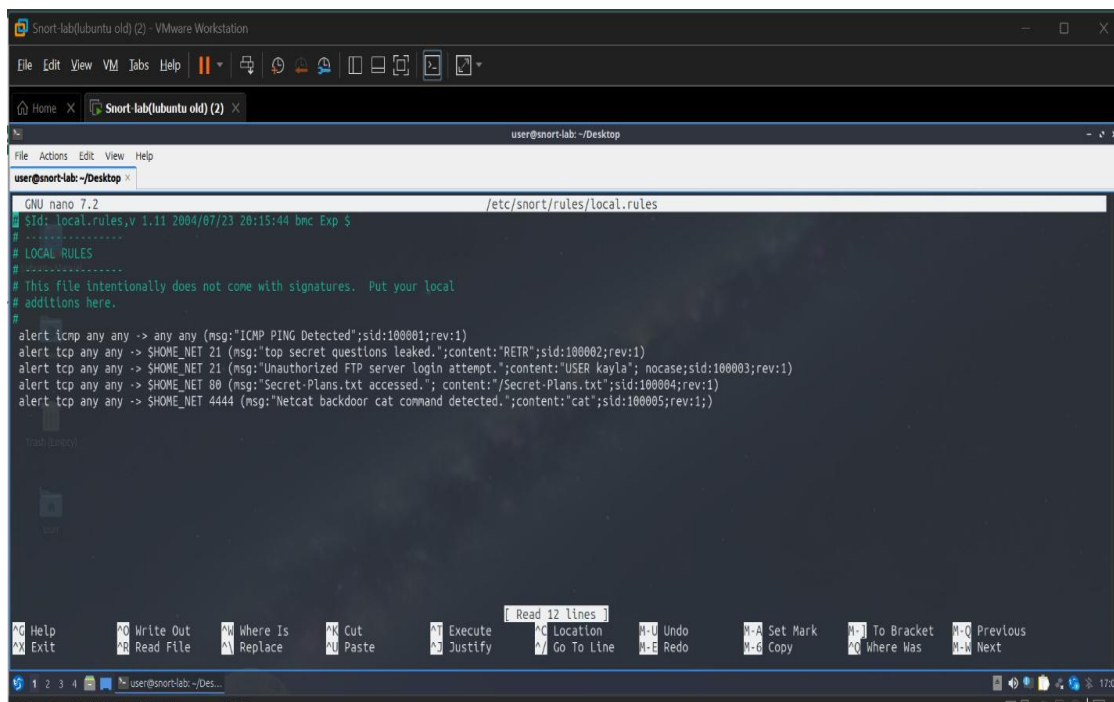
alert tcp any any -> $HOME_NET 4444 (msg:"Netcat backdoor cat command detected.";
content:"cat"; sid:100005; rev:1;)
```

Step 1 — Baseline Connectivity and ICMP Detection

From the Windows attacker machine, the target was pinged (“ping 192.168.129.133”) while Snort ran in IDS mode with the custom rule set loaded (“sudo snort -A console -q -c /etc/snort/snort.conf -i ens33”). Snort immediately generated repeated alerts of the form “[1:100001:1] ICMP PING Detected,” confirming the sensor was actively monitoring the interface and the ICMP rule was functioning correctly for both IPv4 and IPv6 traffic.



Screenshot 1 — Snort running in console mode, successfully validating its configuration and firing “ICMP PING Detected” alerts in response to the Windows host's ping test.



Screenshot 2 — The local.rules file open in nano, showing the five custom signatures written for this lab.

Step 2 — Objective 1: Legitimate FTP File Transfer

A standard FTP session was established to the target and a file (myfile.txt) was retrieved as a baseline, non-malicious transfer. This established a clean reference point for what normal FTP activity looks like before layering in unauthorized attempts.

The screenshot shows a VMware Workstation interface with a single VM named 'Snort-lab(lubuntu old) (2)'. The terminal window is open, showing the following commands and output:

```

user@snort-lab: ~/Desktop
229 Entering Extended Passive Mode (|||56670|)
550 Failed to open file.
ftp> 
ftp> 
ftp> 
ftp> 
ftp> 
ftp> 
ftp> quit
221 Goodbye.
user@snort-lab: ~/Desktop$ ftp 192.168.129.133
Connected to 192.168.129.133.
220 (vsFTPD 3.0.5)
Name (192.168.129.133:user): user
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> get myfile.txt
local: myfile.txt remote: myfile.txt
229 Entering Extended Passive Mode (|||45858|)
150 Opening BINARY mode data connection for myfile.txt (194 bytes).
100% |*****| 194 48.68 KiB/s 00:00 ETA
226 Transfer complete.
194 bytes received in 00:00 (42.68 KiB/s)
ftp>

```

The bottom status bar of the VMware window indicates 'To direct input to this VM, click inside or press Ctrl+G'.

Screenshot 3 (Objective 1) — A successful, legitimate FTP login and file transfer of myfile.txt, used as a baseline for comparison.

Step 3 — Objective 2: Unauthorized FTP Login and Access Attempt

A new local user, kayla, was created on the target, then used to log into FTP and attempt to retrieve sensitive files (secret, secret_file.txt, and myfile.txt). Each of these transfer attempts failed at the FTP layer (“550 Failed to open file”), demonstrating that file permissions correctly blocked the unauthorized user from pulling restricted files — while the FTP login rule (content:"USER kayla") was positioned to flag the attempted access regardless of whether the file transfer itself succeeded.

```

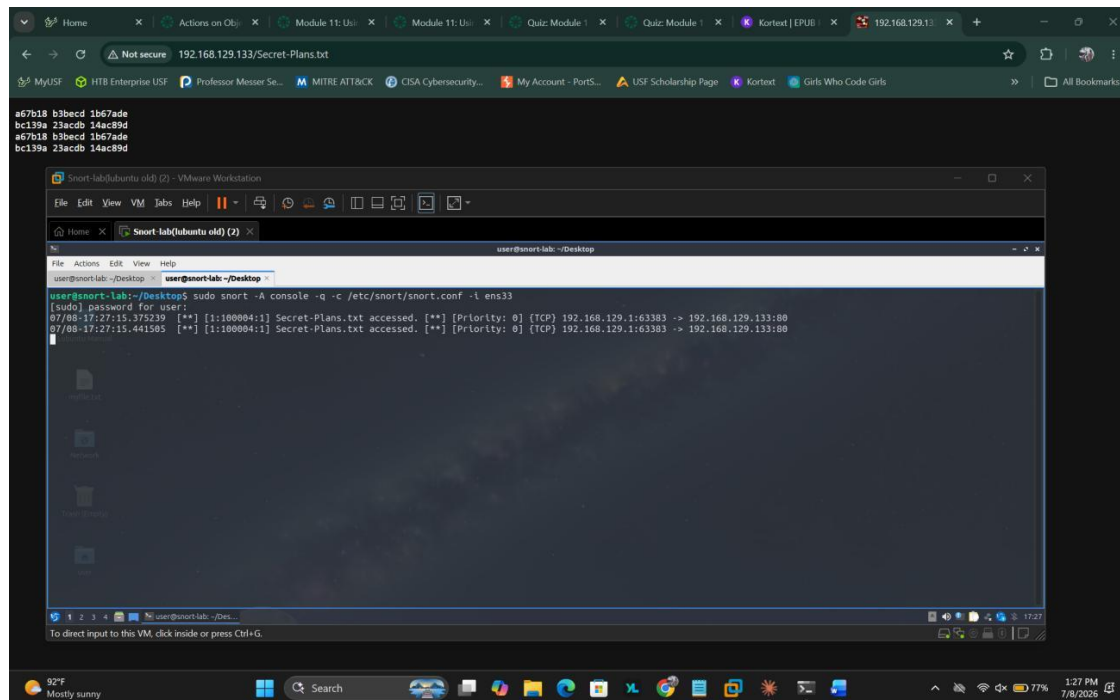
Snort-lab(lubuntu old) (2) - VMware Workstation
File Edit View VM Tabs Help
Home X Snort-lab(lubuntu old) (2) X
user@snort-lab: ~/Desktop
File Actions Edit View Help
user@snort-lab:~/Desktop x
fatal: The user 'kayla' already exists.
user@snort-lab:~/Desktop$ sudo adduser kayla
fatal: The user 'kayla' already exists.
user@snort-lab:~/Desktop$ ftp 192.168.129.133
Connected to 192.168.129.133.
220 (vsFTPd 3.0.5)
Name (192.168.129.133:user): kayla
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> get secret
local: secret remote: secret
229 Entering Extended Passive Mode (|||49014|)
550 Failed to open file.
ftp> get secret_file.txt
local: secret_file.txt remote: secret_file.txt
229 Entering Extended Passive Mode (|||17325|)
550 Failed to open file.
ftp> get secret_file.txt
local: secret_file.txt remote: secret_file.txt
229 Entering Extended Passive Mode (|||33268|)
550 Failed to open file.
ftp> get myfile.txt
local: myfile.txt remote: myfile.txt
229 Entering Extended Passive Mode (|||56670|)
550 Failed to open file.

```

Screenshot 4 (Objective 2) — The newly created “kayla” account logging into FTP and repeatedly failing to retrieve restricted files, each attempt still visible to Snort's login-attempt rule.

Step 4 — Objective 3: Unauthorized Web Access to a Sensitive File

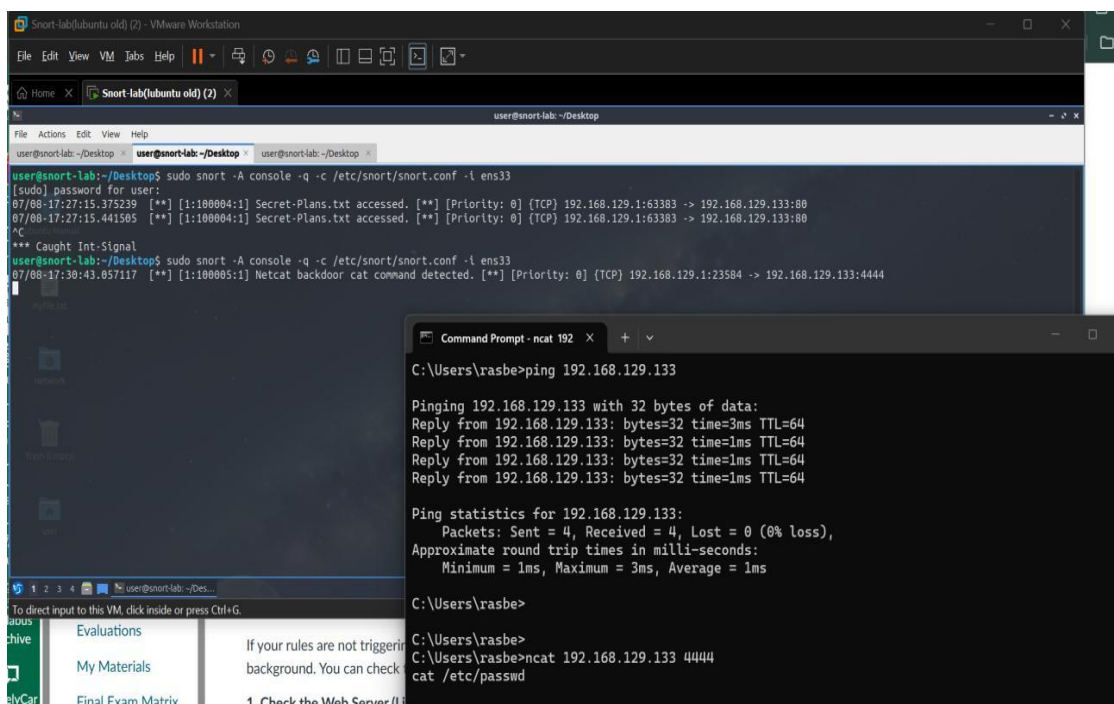
Secret-Plans.txt, a file not meant for public access, was retrieved directly through a browser at <http://192.168.129.133/Secret-Plans.txt>. Snort caught this immediately, logging “[1:100004:1] Secret-Plans.txt accessed” for the matching HTTP request. This confirmed that even quiet HTTP GET requests for sensitive files — the kind an attacker might issue during reconnaissance — can be reliably flagged with a simple content-match rule.



Screenshot 5 (Objective 3) — *Secret-Plans.txt* accessed directly through the browser (top) while Snort logs matching “*Secret-Plans.txt* accessed” alerts in the console (bottom).

Step 5 — Objective 4: Netcat Backdoor Detection

A Netcat listener was simulated on the target at port 4444. From the attacker machine, a connection was established with `ncat` and a command was run to exfiltrate system information (“`cat /etc/passwd`”). Snort detected the backdoor interaction in real time, logging “[1:100005:1] Netcat backdoor cat command detected,” demonstrating how a simple content-based signature on an unusual port can surface classic post-exploitation activity, such as an attacker using a reverse or bind shell to read sensitive system files.



Screenshot 6 (Objective 4) — Snort alerting on the Netcat backdoor's "cat" command while the attacker's ncat session reads /etc/passwd from the target over port 4444.

Key Takeaways

- Signature-based IDS is only as good as the rules behind it. Each custom rule here mapped to a specific, well-understood attacker behavior (recon, credential-based access, sensitive file access, backdoor use).
- Failed actions still matter. Even when the unauthorized FTP downloads failed due to permissions, the attempted login itself was valuable telemetry — a real SOC analyst would want visibility into attempted access, not just successful breaches.
- Port and content context go a long way. A single content:"cat" match on an unexpected port (4444) was enough to catch backdoor usage, showing how unusual ports paired with known command strings are a strong detection heuristic.
- This lab reinforced the full IDS workflow: define a threat, write a rule to detect it, generate matching traffic, and validate the alert fires as expected.